

■ Feature Scaling & Transformation Cheatsheet

(Based on Hands-On Machine Learning, 3rd Edition)

Most ML algorithms work with numerical features and assume that features are comparable in scale and have smooth distributions. Without scaling, large numbers dominate and the model becomes biased. Scaling ensures fair contribution of all features.

■■ Common Scaling Techniques

1. Standardization (Z-score Normalization)

Centers data around 0, sets standard deviation to 1.

Formula: $(x - \text{mean}) / \text{std}$

Use for: Linear Regression, SVM, Logistic Regression, Neural Networks

2. Min-Max Scaling (Normalization)

Rescales values between 0 and 1; sensitive to outliers.

Use for: Neural networks or gradient-based models.

3. Robust Scaling

Centers data using median and IQR to resist outliers.

■ Transformation Techniques

1. Log Transformation

Use for right-skewed data such as income or prices.

2. Power Transformation (Box-Cox / Yeo-Johnson)

Fixes non-normal data automatically.

3. Quantile Transformation

Maps data to a uniform or normal distribution.

■ When to Apply Scaling

Tree-based models (Random Forest, XGBoost) do not require scaling; distance or gradient-based ones do.

■ Full Workflow Example (Python)

```
from sklearn.datasets import fetch_california_housing from sklearn.model_selection import
train_test_split from sklearn.preprocessing import StandardScaler from
sklearn.linear_model import LinearRegression from sklearn.metrics import
mean_squared_error import numpy as np data = fetch_california_housing(as_frame=True) X =
data.data y = data.target X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42) model = LinearRegression() model.fit(X_train, y_train)
```

```
pred = model.predict(X_test) print("RMSE without scaling:",  
np.sqrt(mean_squared_error(y_test, pred))) scaler = StandardScaler() X_train_scaled =  
scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test)  
model.fit(X_train_scaled, y_train) pred_scaled = model.predict(X_test_scaled) print("RMSE  
with scaling:", np.sqrt(mean_squared_error(y_test, pred_scaled)))
```

■ Tips for Mastery

- Fit scalers on training data only, then transform test data using the same scaler. - Combine scaling with pipelines for cleaner workflows. - Visualize feature distributions before and after scaling. - Log transformations require positive data.

■ Practice Datasets

- California Housing (scikit-learn): perfect for regression and scaling. - Wine Quality (UCI Repository): great for outlier handling and PCA. - Diabetes (scikit-learn): ideal for scaling comparisons. - Ames Housing (Kaggle): real-world messy data.

■ Recommended Resources

- Hands-On Machine Learning (Aurélien Géron) - Python for Data Analysis (Wes McKinney) - StatQuest (YouTube) - Kaggle Microcourses: Data Cleaning & Feature Engineering

“Scaling doesn’t change the data’s truth — it just makes sure your algorithms hear each feature at the same volume.”